

Enhancing Programming Education with Open-Source Generative AI Chatbots

Antonia Šarčević *, Ivan Tomićić *, Andrija Merlin *, Marko Horvat *

* University of Zagreb, Faculty of Electrical Engineering and Computing, Department of Applied Computing, Zagreb, Croatia

antonia.sarcevic@fer.hr, ivan.tomicic@fer.hr, andrija.merlin@fer.hr, marko.horvat3@fer.hr

Abstract — This paper describes the development of an Open-Source Generative AI Chatbot, utilizing free Large Language Models (LLM) to enrich the student learning experience for a university course in “Introduction to Programming”. The article aims to provide a step-by-step guide for selecting, fine-tuning, and evaluating available models. As a first step in choosing the appropriate LLM, which provides the most accurate responses while not requiring excessive computing power, the article will cover a discussion of the advantages and disadvantages of local vs. cloud-available models. After selecting a few promising models, the next stage includes fine-tuning LLMs to answer domain-specific questions using a dataset containing essential rules, guidelines, and explanatory content regarding the subject. The crucial aspect of selecting a model was evaluating answers, and in this context, both human and automatic evaluation techniques will be presented. Finally, it is possible to enhance the model performance and accuracy by incorporating Retrieval-Augmented Generation (RAG) techniques and exploring the influence of various factors, such as different vector databases, model temperatures, maximum token lengths, prompt templates, embeddings, repetition penalties, and chunking sizes. Our results show that chatbots have significant potential to improve academic support and learning efficiency, as well as personalized education in general.

Keywords – chatbots; generative models; large language models; natural language processing; education; digital learning

I. INTRODUCTION

The transition to university life always represents a critical stage in a higher education, especially for first-year students dealing with demanding coursework in, what is to them, new and unfamiliar university environments. Freshmen computer science (CS) and computer engineering (CE) students often need help and clarification even with the basic courseware issues, such as the number of points required to achieve a specific grade, or how to install a plugin in the required programming working environment. Although information they require is made available to the students through the official course material and web pages, many of them prefer direct communication with academic staff. On the other hand, teachers and other faculty members are outnumbered by the growing number of students, and the gap seems to be widening as teachers face staff shortages. This ubiquitous problem has sparked the development of many novel digital solutions for support of academic teaching, such as the automated programming assessment systems [1][2].

Recognizing these issues, we developed a user-friendly chatbot that uses open-source artificial intelligence (AI) Large Language Model (LLM) [3] to assist freshmen students with their academic questions on an obligatory first-semester course “Introduction to programming” [4] taught at University of Zagreb, Faculty of Electrical Engineering and Computing. The course teaches fundamental concepts such as data and control structures, functions, arrays, and file handling. It also covers basic software engineering principles such as algorithm design, decomposition, documentation, and the software development lifecycle, giving students the skills to succeed in practical programming and problem-solving.

The goal of this paper is to thoroughly describe the process of theoretical exploration and hands-on training of Large Language Models (LLMs) as part of our project. Our motivation is to share the results and methodologies of the project, thereby facilitating the development of similar AI-based solutions for educational purposes by other researchers. The project was carried out by a joint team of 16 undergraduate and graduate students from the University of Zagreb, Faculty of Electrical Engineering and Computing under supervision of their selected mentor as the fourth author. The team leaders of undergraduate and graduate student teams were the first and second author, respectively.

To be able to successfully complete the project we considered all open LLMs available in October 2023. For injecting domain-specific knowledge into LLM, we explored both fine-tuning and knowledge augmentation mechanisms like Retrieval Augmented Generation (RAG) [5][6]. Our findings favor RAG over fine-tuning for its superior performance in handling existing and new knowledge across different problem domains [7][8].

The remainder of the paper is organized as follows; Section 2 examines LLM selection criteria, focusing on architecture, courseware training data, and performance. Section 3 evaluates local versus cloud LLMs against technical and financial considerations. Section 4 outlines our methodology for training and fine-tuning LLMs with specific datasets. Section 5 assesses model effectiveness using human and automated metrics. Section 6 discusses RAG and its impact on chatbot performance. Finally, we share our findings and recommendations for LLM chatbot deployment in education.

II. MODEL SELECTION

The initial step in our project involved an in-depth exploration of available LLMs – both commercial and

located in the cloud, and free and accessible locally – focusing on understanding the current landscape of high-end language models. The factors we considered are: availability, price per token, model architecture, training data requirements, reported past performance, and fine-tuning capabilities. This comprehensive understanding was crucial in identifying key characteristics that would influence the effectiveness and usability of LLMs. After an exhaustive valuation process, we carefully selected a group of models that demonstrated not only the ability to comprehend but also produced high-quality, contextually relevant responses to courseware questions.

A. Local vs cloud-available LLMs

Our exploration extended to considering models of various sizes, both cloud-available and locally run [3][9]. Multiple cloud-based LLMs were tested, using cloud platforms such as the AI-powered open platform Poe.com [10], alongside locally run LLMs from Hugging Face online repository [11]. This adapted approach allowed us to thoroughly assess the performance of different models in injecting domain-specific knowledge. However, considering our infrastructure limitations, very large models could not be chosen because of the computing power necessary to use them, and cloud-based models were unfit as their usage is often too expensive. In line with that, we selected a group of 10 perspective locally run LLMs of roughly the same size (7 billion parameters) that should be further evaluated to select one optimal model.

B. Training and testing

Following the initial model selection, the hands-on training phase commenced, where our team engaged in a process of training on the official documentation of the “Introduction to Programming” course, an obligatory freshman course at the undergraduate Study Programmes Computing and Electrical Engineering and Information Technology [4]. Specifically, pre-trained LLMs were fine-tuned utilizing available course materials containing instructions for installing local extensions, instructions for command prompt environment, instructions for performing laboratory exercises, and the overall student guide. In total, the LLMs were fine-tuned on a dataset containing 6 documents with 12.828 words.

The objective was to equip each of the 10 selected language models with a foundational understanding of the subject matter, enabling them to generate accurate and contextually relevant responses to user queries. The models were then tested on a set of 20 questions used to narrow the

pool of models. The answers given by the models were evaluated by our student team. It was immediately clear that some models performed significantly worse than others. This brief evaluation was not precisely structured, but a rough review of models that gave us a clear picture of which models should be further tested and which should be discarded. Therefore, we continued our evaluation with a subgroup consisting of the following 5 models:

1. llama-2-7b-chat.Q5_K_M
2. mistral-7b-instruct-v0.1.Q5_K_M
3. mistral-7b-openorca.Q4_0
4. zephyr-7b-alpha.Q5_K_M
5. zephyr-7b-beta.Q5_K_M

To further scrutinize the shortlisted models, we expanded the set of evaluation questions, introducing a spectrum of difficulty levels. We deliberately included questions with contexts outside the model's training data and those containing grammatical errors to assess the robustness of the models. In the end, the full list contained $N = 92$ questions related to the “Introduction to Programming” course. The questions varied in difficulty, ranging from simple, straightforward queries to more complex ones requiring logical reasoning and arithmetic calculations. This diverse set of questions was then fed to the trained LLMs, and their responses were meticulously recorded and evaluated by us based on predefined metrics. In this step, the whole group of 16 students in the team participated in the evaluation, posing all $N = 92$ questions to the selected 5 models. This step is described in more detail in the next subsection. It is important to note that all model names follow the GGML naming convention, with parameter b indicating the model's size in billions of parameters and Q, K, and M corresponding to different weight quantization levels. Quantization is a technique that enables LLMs to run on consumer hardware utilizing only CPU inference.

C. Evaluation

The evaluation phase involved grading the models based on five critical categories: fluency, coherence, relevance, context understanding, and overall quality. Each model received a Likert-scale grade from 1 to 5 in each category, providing a nuanced assessment of its performance. To establish a benchmark for evaluation, we created a list of 3-5 expected answers for each question in the extended set. These expected answers were carefully curated by our team, leveraging the context used for training the LLMs. The results of the evaluation are shown in Table I.

TABLE I. AVERAGE GRADE PER CATEGORY RESULTS FOR EACH OF THE 5 EVALUATED MODELS

Model	Fluency	Coherence	Relevance	Context understanding	Overall quality	Average total
llama-2-7b-chat.Q5_K_M	4.87	4.79	4.15	4.09	3.95	4.35
mistral-7b-instruct-v0.1.Q5_K_M	4.80	4.68	4.34	3.73	3.70	4.25
mistral-7b-openorca.Q4_0	4.92	4.88	4.38	4.44	4.48	4.62
zephyr-7b-alpha.Q5_K_M	4.81	4.71	4.43	3.84	3.67	4.29
zephyr-7b-beta.Q5_K_M	4.87	4.75	4.18	4.15	4.15	4.42

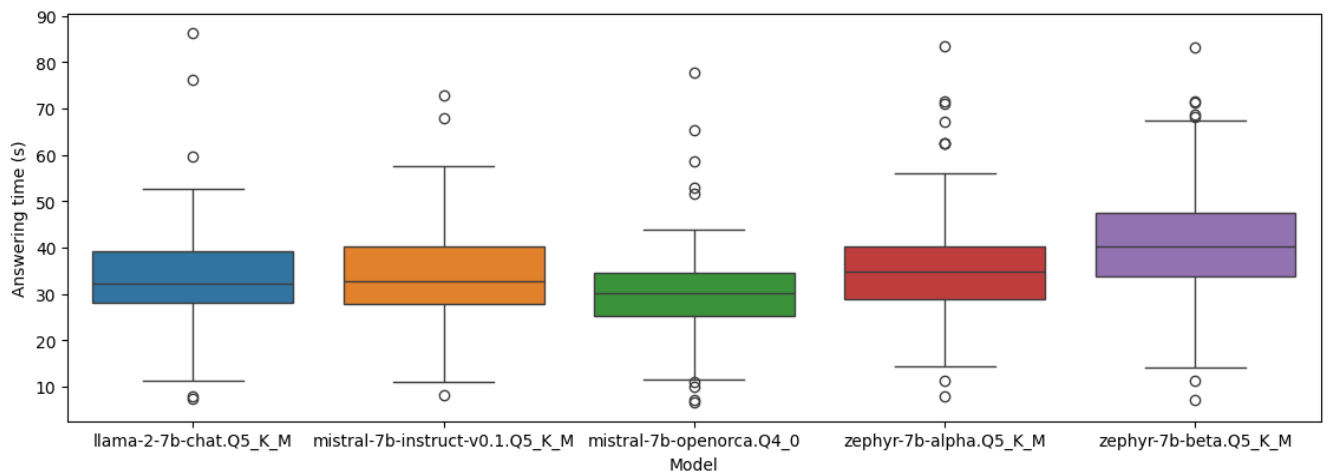


Figure 1. Box-plot with Answering Time per each of the 5 evaluated models

In this phase of our work, we emphasized the importance of depth in the evaluation of these models. Apart from human evaluation, we calculated BLEU [12], ROUGE [13] [14], and diversity scores to provide more objective insight into the effectiveness of the model. The metrics compare the answer generated by the LLM with the corresponding correct answer to the question (our list of 3–5 expected answers). It is good practice to perform these metrics to gain an objective statistical basis for making decisions in a project. In our case, the metrics did not result in high scores, but that can be common when the model's answer does not match the expected one word-for-word.

Considering all evaluation criteria, our choice of the best model was based on the human evaluation, and by that *Mistral-7B-Openorca* significantly outperformed other models.

III. RAG CONFIGURATIONS

The centerpiece of our project was using Retrieval Augmented Generation (RAG) to enhance the performance of natural language processing (NLP) tasks, such as answering questions [5]. By using RAG, our chatbot gains access to more useful information from a database larger than the text it was trained on [5]. Essentially, RAG is a knowledge augmentation mechanism for LLMs, it supplements their inherent capabilities with information retrieved from external knowledge sources [5].

To improve this method and find the optimal configuration we concluded detailed research on this technique and implemented and tested various combinations of different retriever types, chunking sizes and overlaps, number of retrievals, and different embeddings. To understand how this works, a brief explanation of RAG is needed.

A. Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is a methodology used to enhance the capabilities of Large Language Models (LLMs) by incorporating knowledge

from external databases [15]. It addresses challenges faced by LLMs, such as hallucination, outdated knowledge, and non-transparent reasoning processes, by seamlessly integrating domain-specific information. RAG improves the accuracy and credibility of models, particularly in knowledge-intensive tasks, by synergistically merging the intrinsic knowledge of LLMs with the vast, dynamic repositories of external databases. This comprehensive approach allows for continuous knowledge updates and facilitates the integration of domain-specific information [6].

RAG is the process of optimizing the output of a large language model, so it references an authoritative knowledge base outside of its training data sources before generating a response [5]. RAG implements three main components: 1) document chunking, 2) embedding, and 3) searching [16].

Chunking involves dividing documents into parts of arbitrary size [16]. The goal of this process is to achieve a balance between the amount of information and the amount of noise contained in each chunk. Intuitively, choosing a chunk that is too large results in an excessive amount of information, and therefore noise, that cannot be represented as a vector of a specific size. On the other hand, choosing a chunk that is too small leads to dispersed information across many chunks, which may be difficult to manage. In this context, we implemented small chunk sizes of 500 tokens with an overlap of 50 tokens, medium of 1000 tokens with an overlap 200, and large chunks of 2000 tokens with an overlap of 400.

Embedding involves representing the obtained chunks as vectors of a specific size [16]. To convert text into vectors, we use a method called contextual embedding. We wish to retain the word order – sentences can take on different meanings depending on the word order. We also wish to retain the context, because the same word can have a different representation depending on the words it is surrounded by, meaning that sentences containing similar words can have significantly different vectors. In our experimentation, we employed the three following embeddings: 1) all-MiniLM-L6-v2 which maps sentences and paragraphs to a 384-dimensional dense vector space and

offers both speed and quality, 2) Jina embedding which supports longer sequence lengths, and 3) BAAI general

embedding that can map any text to a low-dimensional dense vector.

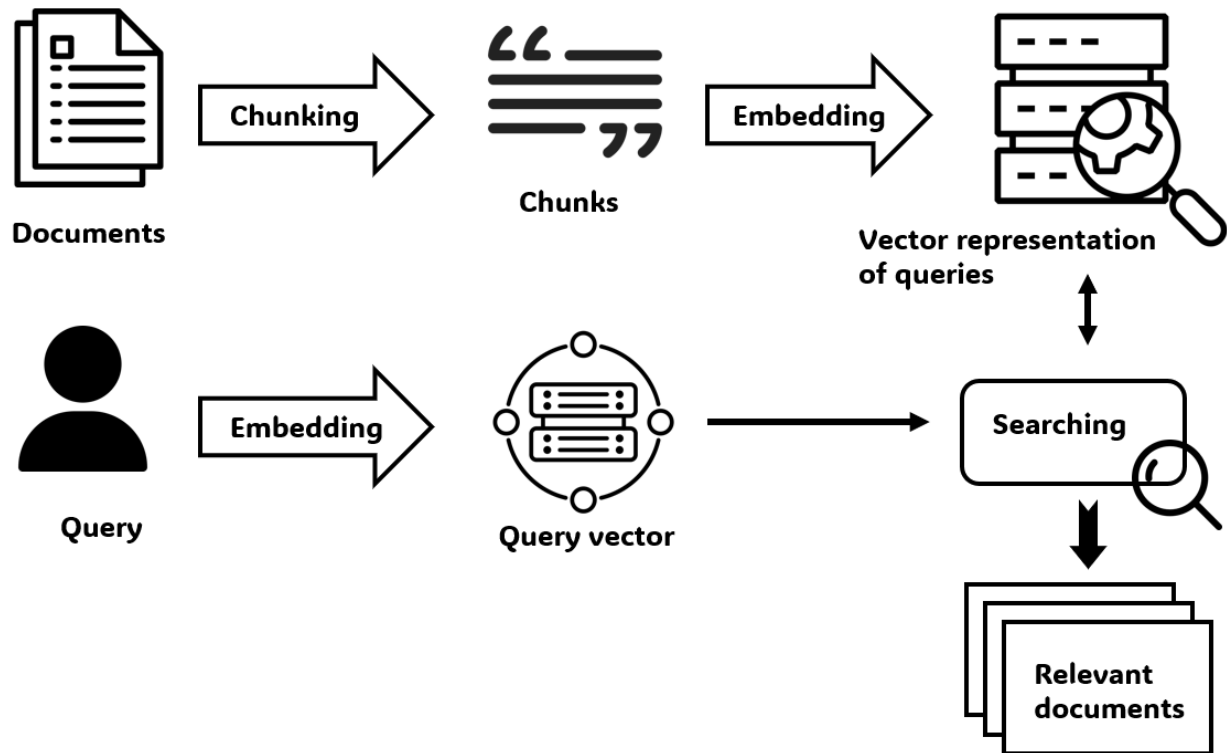


Figure 1. An overview of a typical RAG architecture

In the searching component vector representations of queries and document chunks are used to find relevant documents. Metrics like L2-Norm and cosine similarity measure the distance or similarity between vectors, focusing on properties like length or direction [17][18]. Indices are then employed to search the vector database for relevant documents. Different methods, such as exhaustive search and Voronoi cells, balance search speed, accuracy, and memory usage. Exhaustive search checks all vectors, guaranteeing perfect search quality but scaling with vector number and dimension. Voronoi cells group neighboring vectors to expedite search, with fewer cells to explore compared to vectors. This approach reduces search complexity but relies on efficient indexing. In our work we used L2 distance metric.

To test this method more comprehensively, we evaluated various types of retrievers, including parent retrievers that retrieve a broader set of documents and rerank retrievers that refine the results based on relevance. Within this framework, we explored different combinations of the number of retrievals and the top number of retrievals, such as 5 retrievals with top 3 documents, 7 retrievals with top 2, and 10 retrievals with top 4 documents.

B. Evaluation

In the end, we had 27 configurations with rerank retriever and 2 with parent retriever. Each configuration was given 30 questions, a shortened list of the original group of 92. The answers to each configuration were evaluated by our

team based on new criteria focusing on expression and logic, accuracy and relevance, and overall quality and engagement. This approach not only ensures a thorough analysis of the model's linguistic capabilities but also delves into its contextual understanding and engagement with users.

The expression and logic evaluation criteria provide an assessment of the model's ability to formulate answers clearly and coherently. This analysis, which is divided into five levels from low to high, provides an indication of the model's ability to convey information effectively.

Accuracy and relevance as further evaluation measures relate directly to the content performance of the model. The five-level rating, which ranges from low to high, takes into account the accuracy of the information provided, its relevance to the prompt, and the model's understanding of the content.

The third criterion, overall quality, and engagement, combines the ratings of expression, logic, accuracy, and relevance. This approach provides a comprehensive assessment of the language model's response, considering both technical accuracy and the ability to captivate and engage the reader.

We improve the depth of our analysis of the chosen model while demonstrating our commitment to understanding the language model's performance in real-world scenarios. It conforms to best practices in model testing and validation and provides valuable insights for further refinement and optimization.

C. Results

Although different configurations give similar answers to simpler questions in the provided example, we can see that parameter selection can significantly affect the accuracy of answers to more complex questions that require deeper understanding and broader context. As a result, the configuration with the following parameters had the best human ratings:

- Chunk size: 1000
- Chunk overlap: 200
- Number of retrievals: 5
- Maximum number of retrievals: 3
- Embedding: all-MiniLM-L6-v2

Averaging a total mean rating of 4.13 with an average duration per question of 66.55 seconds¹.

It is important to note that the context size significantly affects the response time. For example, the fastest configuration with parameters 500, 50, 7, 2, all-MiniLM-L6-v2 had an average response time of 29.96 seconds and an overall human rating of 3.5, while the slowest configuration with parameters 2000, 400, 10, 4, Jina embedding had an average response time of 235.48 seconds per response, with an overall human rating of 3.53. Both configurations mentioned above used the rerank retriever.

IV. CONCLUSION AND FUTURE WORK

In the described research, as part of our project, we thoroughly investigated the available LLMs, taking into account factors such as model architecture, training data and fine-tuning capabilities. This comprehensive understanding helped us select a model that not only understands but also provides high-quality, contextualized answers. In addition, we recognized the importance of overcoming challenges such as hallucinations and outdated knowledge, which led us to integrate the RAG methodology into our project. By incorporating knowledge from the external dataset, RAG enhanced the accuracy and credibility of our chatbot, especially for knowledge-intensive tasks. Specifically, we utilized available materials for the course "Introduction to Programming" containing instructions for installing local extensions in the working environment, instructions for command prompt environment, instructions for performing course laboratory exercises, and the overall student guide for the course. This external dataset consisted of 6 documents with 12.828 words, in total.

The process began with model selection, followed by a hands-on training phase in which the team fine-tuned LLMs using the aforementioned official documentation from "Introduction to Programming" course. Following training, the models were evaluated with 20 questions, yielding a preliminary assessment that revealed significant performance differences between them. Five models were chosen for further testing based on the results of the initial evaluation. After this step, the evaluation was expanded to

include a more diverse set of 92 questions of varying complexity, all designed to test the models' robustness and understanding. These questions, some of which contained intentional grammatical errors or contexts unrelated to the training data, were presented to the LLMs. A team of 16 students thoroughly analyzed the responses, using predefined metrics to assess the shortlisted models' performance in detail.

Models were evaluated in five key areas: fluency, coherence, relevance, context understanding, and overall quality, with ratings ranging from 1 to 5 for each criterion. To benchmark, 3–5 expected answers per question were prepared using the LLMs' training context.

Based on comprehensive evaluation criteria, including human assessment, *the Mistral-7B-Openorca model was found to be significantly superior to the others.*

In the future, we plan to extend our chatbot project to several subjects or even to the entire college curriculum. By extending the functionality of the chatbot, we hope to provide students with comprehensive support and assistance throughout their studies. This future direction is consistent with our overall goal of using technology to improve learning outcomes and help students succeed in higher education. The potential success of the chatbot in assisting students emphasizes its ability to be used in a variety of academic disciplines, not only in computer engineering. Using the basic framework created through theoretical research and practical exercises, we can customize and refine the chatbot's capabilities to meet the specific needs and requirements of different subjects.

Finally, in continuation of our previous research [19][20][21], we would like to integrate the developed AI-based chatbot into an adaptive e-learning intelligent agent that can be used in any learning environment that enables remote sensing of students' physiological signals. The intelligent agent evaluates students' engagement and learning curves by analyzing their affective states, behavioral parameters, and psychophysiological features using unobtrusive video surveillance and machine learning. The described system has the potential to personalize learning and improve pedagogical outcomes, which is critical for creating future e-learning environments.

ACKNOWLEDGMENT

We sincerely thank all members of the student and graduate teams for their invaluable contributions to this project. Luka Babić, Jure Franjković, Vedran Knežević, Ivan Lisica, Andrija Merlin, Fran Sipić, Josip Šare, Lora Šušić, Stela Troskot, in the undergraduate team, and Matej Božić, Lovro Joja, Klara Krmpotić, Luka Mujagić, and Katarina Virkes, in the graduate team, contributed to the successful outcome of the project with their hard work and dedication. The team leaders of the undergraduate and graduate teams were Antonia Šarčević and Ivan Tomičić, respectively.

¹ All time measurements mentioned in this document were obtained using a personal computer equipped with an AMD Ryzen7 5800H processor, 16 GB RAM and an

NVIDIA GeForce GTX 1650 GPU and running the 64-bit Microsoft Windows 11 operating system.

REFERENCES

- [1] I. Mekterović, L. Brkić, and M. Horvat, "Scaling Automated Programming Assessment Systems," *Electronics*, vol. 12, no. 4, p. 942, 2023.
- [2] S. Wang, M. Wen, B. Lin, H. Wu, Y. Qin, D. Zou, et al., "Automated patch correctness assessment: How far are we?," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, pp. 968-980, December 2020.
- [3] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang, and W. Ye, "A survey on evaluation of large language models," *ACM Transactions on Intelligent Systems and Technology*, 2023.
- [4] Introduction to programming, <https://www.fer.unizg.hr/en/course/itp> (last accessed 12 February 2024)
- [5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.T. Yih, T. Rocktäschel, and S. Riedel, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459-9474, 2020.
- [6] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, et al., "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.
- [7] O. Ovadia, M. Brief, M. Mishaeli, and O. Elisha, "Fine-Tuning or Retrieval? Comparing Knowledge Injection in LLMs", *arXiv [cs.AI]*. 2024.
- [8] P. Zhang, S. Xiao, Z. Liu, Z. Dou, and J. Y. Nie, "Retrieve anything to augment large language models," *arXiv preprint arXiv:2310.07554*, 2023.
- [9] S. Minaee, T. Mikolov, N. Nikzad, M. Chenaghlu, R. Socher, X. Amatriain, and J. Gao, "Large Language Models: A Survey," 2024. [Online]. Available: [arXiv:2402.06196](https://arxiv.org/abs/2402.06196) [cs.CL]. DOI: 10.48550/arXiv.2402.06196
- [10] Poe.com, <https://poe.com> (last accessed 12 February 2024)
- [11] S. M. Jain, "Hugging Face," in *Introduction to Transformers for NLP: With the Hugging Face Library and Models to Solve Problems*, Berkeley, CA: Apress, 2022, pp. 51-67.
- [12] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, 'Bleu: a Method for Automatic Evaluation of Machine Translation', in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 2002, pp. 311-318.
- [13] K. Ganesan, "Rouge 2.0: Updated and improved measures for evaluation of summarization tasks," *arXiv preprint arXiv:1803.01937*, 2018.
- [14] Y. Graham, "Re-evaluating automatic summarization with BLEU and 192 shades of ROUGE," in *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pp. 128-137, September 2015.
- [15] L. Hu, Z. Liu, Z. Zhao, L. Hou, L. Nie, and J. Li, "A survey of knowledge enhanced pre-trained language models," *IEEE Transactions on Knowledge and Data Engineering*, 2023.
- [16] P. Finardi, L. Avila, R. Castaldoni, P. Gengo, C. Larcher, M. Piau, P. Costa, and V. Caridá, "The Chronicles of RAG: The Retriever, the Chunk and the Generator," *arXiv preprint arXiv:2401.07883*, 2024.
- [17] W. Hu, L. Wu, M. Jian, Y. Chen, and H. Yu, "Cosine metric supervised deep hashing with balanced similarity," *Neurocomputing*, vol. 448, pp. 94-105, 2021.
- [18] E. Deza, M. M. Deza, M. M. Deza, and E. Deza, *Encyclopedia of Distances*, Springer Berlin Heidelberg, pp. 1-583, 2009.
- [19] M. Horvat and T. Jaguš, "Emerging opportunities for education in the time of covid-19-adaptive e-learning intelligent agent based on assessment of emotion and attention," in *Proceedings of the Central European Conference on Information and Intelligent Systems*, Faculty of Organization and Informatics Varazdin, pp. 203-210, 2020.
- [20] D. Doljanin, L. Pranjić, L. Jelečević, and M. Horvat, "Adaptive intelligent agent for e-learning: First report on enabling technology solutions," in *2021 44th International Convention on Information, Communication and Electronic Technology (MIPRO)*, pp. 1690-1694, September 2021, IEEE.
- [21] M. Horvat, D. Doljanin, and T. Jaguš, "Quantitative measures for classification of human upper body posture in video signal to improve online learning," in *AIP Conference Proceedings*, vol. 2570, no. 1, August 2022, AIP Publishing.